S2N XSS Security Report

Generated at: 2026-05-13T17:01:55.899003Z

1. Executive Summary

This report was generated from an actual S2N scan result and describes confirmed XSS findings.

Primary Finding	Cross-Site Scripting Detected
Primary Severity	HIGH
Primary Confidence	FIRM
Target URL	http://host.docker.internal:4280/vulnerabilities/xss_r/?name=%3Cscript%3Ealert(1)%3C%2Fscript%3E
Affected Parameter	name
Primary Payload	<script>alert(1)</script>
Total Findings	1
Severity Counts	{'HIGH': 1}
Total Requests	1
Source Scanner	s2n
Source Plugin	xss

> The finding is confirmed because the scanner observed unsafe payload handling through HTTP reflection or browser execution evidence.

2. All Findings Summary

The S2N scan detected the following XSS finding records. The following sections summarize each finding in a vertical detail format and then analyze the primary finding.

2.1 Finding 1

ID	xss-1
Severity	HIGH
Confidence	FIRM
Target Path	/vulnerabilities/xss_r/?name=...
Parameter	name
Payload	<script>alert(1)</script>
Verification	payload_reflected
Reflected	true

3. Representative Finding Detail

ID	xss-1
Title	Cross-Site Scripting Detected
Vulnerability Type	XSS
XSS Type	reflected
CWE	CWE-79
OWASP Category	Injection / Cross-Site Scripting
HTTP Method	GET
Parameter	name
Injection Context	url_parameter
Reflection Detected	true
Reflected Value	<script>alert(1)</script>
Verification	payload_reflected
Risk Score	76
Likelihood	High
Impact	Medium

Description

User-controlled input was reflected or executed in an unsafe browser/HTML context. This may allow JavaScript execution in a victim browser.

4. Evidence

Raw Evidence

```
XSS evidence confirmed by scanner; payload=<script>alert(1)</script>;
verification=payload_reflected; status_code=200; method=GET; parameter=name;
discovery_mode=direct_url_validation;
```

Response Snippet

```
XSS evidence confirmed by scanner; payload=<script>alert(1)</script>;
verification=payload_reflected; status_code=200; method=GET; parameter=name;
discovery_mode=direct_url_validation;
```

5. XSSAgent Judgement

Task	xss_reflection_confirmation
Should Run	true
Context Known	true
Selected Plugin	-
Severity	HIGH
Confidence	97
Fallback	xss_stored_confirmation
Reason	XSS evidence confirmed by scanner.

6. Risk & Remediation

Risk

Severity	HIGH
Risk Score	76
Likelihood	High
Impact	Medium
OWASP Likelihood Score	9
OWASP Impact Score	5
Scoring Model	risk_score_first_evidence_and_rag_supported
Score Scale	0-100 normalized report score. Severity is assigned from CVSS-inspired qualitative bands; this is not a formal CVSS vector.
Scanner Reported Severity	HIGH
Business Impact	An attacker may execute arbitrary JavaScript in a victim browser, steal session data, perform actions as the user, or manipulate page content.

Scoring Basis

• Payload reflection was observed, increasing exploit likelihood.

• The payload contains script-capable input, increasing client-side impact.

• The evidence indicates executable JavaScript risk, not only passive reflection.

• The vulnerable input is reachable through a GET URL parameter.

• Scanner confidence is firm/high, so the finding is treated as verified evidence.

• Risk score is calculated first from OWASP-style likelihood and impact scores using 45% likelihood and 55% impact weighting: 76/100.

• Severity is then assigned from the score range: Critical 90-100, High 70-89, Medium 40-69, Low 10-39, Info 0-9.

• Hybrid RAG matched official references used as supporting rationale: OWASP - OWASP Risk Rating Methodology, OWASP - OWASP XSS Prevention Cheat Sheet, FIRST - CVSS v3.1 Qualitative Severity Rating Scale, PortSwigger - PortSwigger Web Security Academy: Reflected XSS.

Recommended Remediation

1. Apply context-aware output encoding before inserting user-controlled input into HTML.

2. Validate and constrain user input on the server side.

3. Avoid directly reflecting raw request parameters into responses.

4. Use safe DOM APIs such as textContent instead of innerHTML where possible.

5. Add Content Security Policy as a defense-in-depth control.

6. Re-test the affected parameter with the same payload after remediation.

7. Internal Knowledge Retrieved by ChromaDB

7.1 Recommended Fix Summary

Source	docs/security_guides/xss_remediation.md
Retriever	chroma
Retrieval Score	0.8684

Recommended Fix

• Apply context-aware output encoding.
• Escape HTML body output using HTML entity encoding.
• Escape HTML attribute values.
• Avoid inserting user input directly into JavaScript context.
• Prefer safe DOM APIs such as textContent instead of innerHTML.
• Validate and normalize input on the server side.
• Use Content Security Policy as a defense-in-depth control.

7.2 Reflected XSS Explanation

Source	docs/security_guides/xss_remediation.md
Retriever	chroma
Retrieval Score	0.8677

Reflected XSS

Reflected XSS occurs when user-controlled input is included in an HTTP response without proper output encoding. If the browser interprets the reflected input as HTML or JavaScript, an attacker may execute script in the victim's browser context.

7.3 HTML Body Context Guidance

Source	docs/security_guides/xss_remediation.md
Retriever	chroma
Retrieval Score	0.8622

HTML Body Context

When untrusted input is rendered inside the HTML body, apply HTML entity encoding before rendering the value.

For example, <script>alert(1)</script> should be rendered as <script>alert(1)</script>.

8. Official Reference Mapping

8.1 OWASP - OWASP Risk Rating Methodology

Source	https://owasp.org/www-community/OWASP_Risk_Rating_Methodology
Source Mode	official_catalog
Score	29.9

OWASP describes application security risk as a combination of likelihood and impact. It recommends estimating both dimensions and combining them into an overall severity. Recommended use: Use this reference as the primary rationale for evidence-based likelihood and impact scoring. Official URL: https://owasp.org/www-community/OWASP_Risk_Rating_Methodology Source URL: https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

8.2 OWASP - OWASP XSS Prevention Cheat Sheet

Source	https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
Source Mode	official_catalog
Score	24.7

OWASP recommends context-aware output encoding and layered defensive controls to prevent Cross-Site Scripting. Different output contexts such as HTML body, HTML attributes, JavaScript, CSS, and URL contexts require different encoding rules. Recommended use: Use this reference as the primary remediation guideline for XSS output encoding. Official URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html Source URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

8.3 FIRST - CVSS v3.1 Qualitative Severity Rating Scale

Source	https://www.first.org/cvss/v3.1/specification.document
Source Mode	official_catalog
Score	24.0

FIRST CVSS v3.1 defines qualitative severity bands for 0.0-10.0 scores: Low, Medium, High, and Critical. This project uses those bands as a familiar reporting vocabulary, not as a full CVSS vector calculation. Recommended use: Use this reference to explain why report scores are grouped into Low, Medium, High, and Critical bands. Official URL: https://www.first.org/cvss/v3.1/specification.document Source URL: https://www.first.org/cvss/v3.1/specification.document

8.4 PortSwigger - PortSwigger Web Security Academy: Reflected XSS

Source	https://portswigger.net/web-security/cross-site-scripting/reflected
Source Mode	official_catalog
Score	21.0

PortSwigger describes reflected XSS as occurring when an application receives data in an HTTP request and includes that data within the immediate response in an unsafe way. Recommended use: Use this reference to explain reflected XSS behavior and testing methodology. Official URL: https://portswigger.net/web-security/cross-site-scripting/reflected Source URL: https://portswigger.net/web-security/cross-site-scripting/reflected

8.5 CWE - CWE-79: Improper Neutralization of Input During Web Page Generation

Source	https://cwe.mitre.org/data/definitions/79.html
Source Mode	official_catalog
Score	20.7

CWE-79 classifies Cross-Site Scripting as improper neutralization of input during web page generation. Recommended use: Use this reference to classify the weakness and map the finding to CWE-79. Official URL: https://cwe.mitre.org/data/definitions/79.html Source URL: https://cwe.mitre.org/data/definitions/79.html

8.6 MDN - MDN Content Security Policy Guide

Source	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
Source Mode	official_catalog
Score	20.4

MDN describes Content Security Policy as a defense-in-depth mechanism that can help reduce the impact of XSS by controlling script execution sources. Recommended use: Use this reference as a defense-in-depth recommendation, not as a replacement for output encoding. Official URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP Source URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP

9. Retest Recommendation

Re-run the S2N XSS plugin against the affected endpoint after remediation.

Target URL	http://host.docker.internal:4280/vulnerabilities/xss_r/?name=%3Cscript%3Ealert(1)%3C%2Fscript%3E
Parameter	name
Payload	<script>alert(1)</script>
Expected Secure Result	The payload should be encoded or rejected and must not execute as JavaScript.

Retest Steps

1. Send the same GET request to the affected endpoint with the original payload.

2. Verify that < and > are encoded as < and >, or that the input is rejected.

3. Open the response in a browser and confirm that no JavaScript alert or script execution occurs.

4. Re-run the S2N XSS plugin and confirm that the finding count for this endpoint is zero.